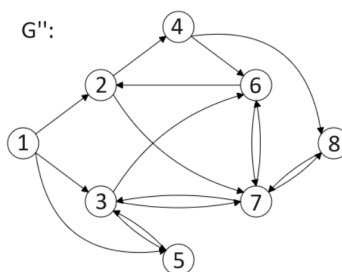
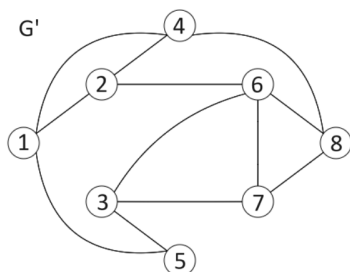


NAME:
Neptun:

- 1) a) Present the adjacency list representation of both graphs.
- b) Present the adjacency matrix representation of both graphs.
- c) Give an example of a simple loop (cycle) of length 5 in G'' .
- d) Remove at most 10 edges from G'' to obtain a directed tree. Give a textual representation of this tree.



- 2) Present breadth-first search (BFS) with vertex 3 as source vertex on the directed graph given by the following textual representation:

$1 \rightarrow 2; 4.$ $4 \rightarrow 2.$
 $2 \rightarrow 5.$ $5 \rightarrow 7.$
 $3 \rightarrow 5; 6.$ $6 \rightarrow 4.$

(When you have a choice during the algorithm, always choose the vertex with smaller index.)

- a) Draw a graphical representation of the given graph.
- b) Illustrate the running of the algorithm by filling this table:

changes of d						ex- panded vertex : d	$Q :$ Queue $\langle \quad \rangle$	changes of π					
1	2	3	4	5	6			1	2	3	4	5	6
						final d and π values							

- c) Which edges will be edges of the BFS tree? Label them on your picture, or give a list of these edges.

- 3) Run depth-first search algorithm (DFS) on the directed graph given by adjacency matrix A .

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

(If you have a choice during the algorithm, then always select the vertex with smallest index.)

a) Draw a graphical representation of the graph. Illustrate the tree edges on this picture with double lines (or an extra color) and write the (discovery time/finishing time) pair beside every vertex.

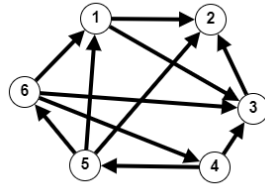
b) List the backward edges, forward edges and cross edges!

Backward (B) edges:

Forward (F) edges:

Cross (C) edges:

4) Determine the transitive closure of the following graph:



Give your answer as a matrix T .

You can use either of the methods we have seen in practice.

a) If you use the modified BFS algorithm, then present your answer by giving the tables.

b) If you use the dynamic programming algorithm (the second algorithm we have seen in practice), then give matrices $T^{(0)}, T^{(1)}, T^{(2)}, \dots$ that are produced by this algorithm.

5) Demonstrate the algorithm TopologicalSort on the DAG below. Show the ordering of vertexes produced by the algorithm. Assume that in the indeterministic cases procedure DFS considers the vertices in increasing order.

a	→	b; e.	b	→	c; d; e.
c			d	→	c; e; f.
e	→	f.	f		

6) Construct an algorithm (given in structogram format) which solves the following problem: given a directed graph G , we would like to "add" a new vertex to it which is connected to all other vertexes. The algorithm receives G with adjacency list representation $A/1 : Edge * [n]$, and it should return a new adjacency list $B/1 : Edge * [n + 1]$ which is the representation of the new graph.

7) Construct an algorithm which solves the following problem: for a given graph G on n vertices, a given vertex $s \in G.V$ and a given natural number K , we would like to count the number of vertices of G that are at distance at most K from vertex s . Try to construct an algorithm which does not traverse through all vertices. Present your algorithm in structogram or pseudocode format.